联合作战指挥多 Agent 系统 · 汇报稿

> 时长约 6 分钟。括号内是提示，不用念。照读即可，每节控制在标注时间内。

开场（40 秒）

我汇报的是联合作战指挥多 Agent 系统的设计方案。

一句话说清楚它干什么：**把用自然语言描述的作战问题，可靠地转换成一份有全局约束、数字可追溯、冲突可以裁决的结构化方案。**

请注意这里有三个关键词——**有约束、可追溯、能裁决**。这不是一个陪人聊军事话题的机器人，它的产出是要能被检查、能被追问、能被复盘的。

一、核心玩法（80 秒）

它的核心玩法是**"总指挥加三军参谋"的角色分工，不是让一个大模型包打天下。**

举两个例子：

第一种情况，用户问"陆军现在多少人、多少装备"。这个问题系统只派陆军参谋一个人去查数据库，查完直接回答，**海军、空军、总指挥都不惊动**。又快又省。

第二种情况，用户给一个复杂作战背景，比如一次联合登陆。这时候才由总指挥出场：拆解任务、下发全局约束，然后海陆空三个参谋**同时**各自拟方案；三个方案出来以后，系统先检测他们有没有抢同一批资源——运输机架次、火力支援、时间窗口这些都是共享的；**没冲突就直接合成，有冲突就让冲突的双方开一轮协商，最多三轮，谈不拢由总指挥强制裁决。**

我特别说明一下为什么要这样设计。如果让一个大模型一口气写完整个方案，会有三个问题躲不掉：**数字是它"说"出来的，不是查出来的**；**条件它自己会改**，今天说用一个旅明天改成两个旅；**三军抢资源它看不出来**，因为它是一口气写下来的，不存在三方对账的动作。我们这套分工，本质上是把这三个漏洞用流程堵住。

二、现在做到哪一步（70 秒）

我如实汇报进度，分三层：

第一层，架构和规则设计，百分之百完成。 四个角色怎么分工、两条链路怎么走、八条硬规则、五条收敛判据、七条通信契约，全部定稿，没有悬而未决的架构问题。

第二层，控制流原型，大约六成，而且是实测过的。 我用模拟模型把整条链路跑通了：十二项链路验证加十四项规则自测，全部通过。具体跑通了什么——领域外提问会拒答；单域问题走快通道只派一个参谋；三个参谋能并行出方案、能协商、能仲裁；还有四类故障，方案超限、产物残缺、约束矛盾、某个参谋失联，系统都能正确处置，**不会崩。**

第三层，产品代码，一行都还没写。 现在目录里只有文档、图和测试脚本。

我要主动说明一句：**原型验证的是"控制流"，不是"语义"。** 也就是说，链路跑得通这件事被证明了，但真模型下参谋拟出来的方案质量如何、真实兵力数据接进来怎么样，还没验。这正是下一步要优先解决的。

三、接下来怎么走（70 秒）

分五期推进，每期单独验收，不做最后一锤子买卖。

第一期就是本轮：**建真实兵力池，接线上模型，跑通最小闭环**，判据很简单——"陆军多少人"这个问答得对，而且数字能追到库里。

第二期做协商落盘和硬规则执行；第三期做多轮会话和增量重规划；第四期做本地部署大模型，同时做集成和韧性；第五期才做 MCP 外部服务和前端页面。

这里有两个顺序上的安排，我说明一下理由：

为什么先接线上模型、后面才本地部署？ 因为要先用最强的模型把"参谋到底能不能干好这个活"验证清楚。否则一旦效果不好，就会出现"是模型不行还是工程有问题"的互相甩锅。本地部署放在第四期，到时候只需要做能力回归，不用推翻设计。

为什么前端放最后？ 因为底层能力没定型就先做界面，界面一定返工。所以一到四期全是后端，用命令行和日志验收，第五期才上页面。

四、风险（60 秒）

三条主要风险：

第一，最大的风险是中途失控——目标范围不小，而验收标准定的是"链路跑通"。对策就是刚才说的分期验收，每期有独立判据，一期不通过不往下走。

第二，语义层还没验证。 真模型下参谋方案质量未知。这是第一期要第一时间打掉的问号。

第三，本地部署之后方案质量可能下降。 换成本地模型能力会掉一截，对策是拿同一套测试用例做回归，不达标就调整本地模型规格，或者保留线上加本地的混合模式。

五、需要什么（40 秒）

四样东西：**线上大模型的 API 账号**（主力加轻量双档，双档可以省掉大约八成的辅助调用成本）；**兵力池的种子数据**（三份 JSON，字段结构我们出，数据请业务方填）；第四期本地部署的算力（等第二期之后评估）；第五期的前端人力。

收尾（20 秒）

以上就是设计方案和推进计划。四项关键决策——先做最小可用、先线上后本地、验收口径是链路跑通、先底层后前端——都已经确定，写进了项目书。

如果领导认可，我下一步就开工第一期，先把真实兵力池建起来、线上模型接上，让"陆军多少人"这个问答得准、数字可追溯。有了这个，后面四期才有可信的地基。

附：领导可能问的问题，提前准备好怎么答

问：这跟直接接一个大模型有什么区别？ 答：三个硬区别——数字全部查库、带来源可审计，不是模型心算；全局约束只能总指挥下发，参谋不能自己改条件；三军抢资源会被检测出来并裁决。还有两条：领域外的问题明确拒答，不做通用助手；支持多轮改条件，改一个条件只重算受影响的参谋。

问：多久能做完？ 答：我不给拍脑袋的工期，只给判据。第一期就是"陆军多少人答得对、数字可追溯"，这一条做到了第一期就完成。工期取决于投入的人力，您可以根据各期判据自己排。

问：现在一行代码都没有，前面时间花在哪了？ 答：花在设计决策和验证上。这套系统的难点不在写代码，在于定清楚什么情况下只叫一个参谋、什么情况下三个一起上、冲突了谁说了算。这些问题如果边写边想，返工代价很大。而且我们已经不是纸面设计——控制流跑通了，十二项加十四项测试全过，过程中还抓出了几个真 bug，比如参谋节点会自己给自己派活跑成死循环。这些坑现在踩掉，比写完三千行代码再踩便宜得多。

问：为什么 MCP 和前端都放最后？ 答：MCP 是接情报、气象这类外部数据的，属于能力扩展，主链路不依赖它，先做没有收益。前端放最后是怕底层没定型被界面绑死。这两个都不影响系统能不能用。

问：为什么要先线上后本地？ 答：为了把"模型能力"和"工程问题"分开验证。先线上用强模型证明参谋确实能干好活，再换本地，那时候如果效果下降，责任清楚——就是模型规格的问题，调规格或者走混合模式，不用怀疑设计。

问：数据从哪来？ 答：我们自己定字段结构——单位、人数、装备型号数量、驻地这些，先出三份 JSON 模板，请业务方填数，然后灌进 SQLite。参谋只能通过查询拿数字，不能自己编。

问：能不能保证数字不出错？ 答：分两层。架构上做了硬约束：所有数字必须来自 SQL 查询并带来源标记，没有来源的数字通不过校验——这一条是机器判的，不是提示词要求的。但"保证零差错"是更强的承诺，需要真实数据接进来之后做一轮核对才能下结论，目前的验收口径定的是链路跑通。

问：做出来到底能干什么用？ 答：定位是筹划层面的决策支持，不是实战指挥。用户提一个作战背景，系统出一份带兵力编成、行动时序、风险和备选的方案，而且每一步都能追问"这个数字哪来的""为什么这么定"。领域外的问题它会拒答，真实情报和实战指挥它不做。